1.1: Organização e Descrição da Telemetria

```
# Comandos para importar as bibliotecas em Python
import pandas as pd
import numpy as np
from datetime import datetime, timedelta
import os
```

```
# Configurações
n_rows = 500
start_time = datetime.now()
anomaly_rate = 0.1

# Gerar timestamps sequenciais
timestamps = [start_time + timedelta(minutes=i) for i in range(n_rows)]

# Gerar dados normais dentro das faixas
data = {
    'timestamp': timestamps,
    'internal_temp_c': np.random.uniform(18, 35, n_rows),
    'external_temp_c': np.random.uniform(-5, 30, n_rows),
    'battery_voltage_v': np.random.uniform(46, 52, n_rows),
    'battery_current_a': np.random.uniform(20, 120, n_rows),
    'battery_soc_percent': np.random.uniform(60, 100, n_rows),
    'battery_capacity_ah': np.random.uniform(80, 120, n_rows),
    'power_load_kw': np.random.uniform(5, 25, n_rows),
    'energy_loss_percent': np.random.uniform(2, 8, n_rows),
    'tank_pressure_bar': np.random.uniform(95, 145, n_rows),
    'structural_integrity': [1] * n_rows,
    'critical_modules_status': [1] * n_rows,
    'telemetry_link_status': [1] * n_rows,
    'estimated_autonomy_min': np.random.uniform(45, 180, n_rows),
    'anomalia_inserida': ['nao'] * n_rows
}

df = pd.DataFrame(data)

# Inserir anomalias (3% das linhas)
n_anomalies = int(n_rows * anomaly_rate)
anomaly_indices = np.random.choice(df.index, n_anomalies, replace=False)

for idx in anomaly_indices:
    df.at[idx, 'anomalia_inserida'] = 'sim'
    anomaly_type = np.random.choice(['temp', 'battery', 'pressure', 'integrity', 'modules', 'link'])

    if anomaly_type == 'temp':
        df.at[idx, 'internal_temp_c'] = np.random.uniform(45, 70)
    elif anomaly_type == 'battery':
        df.at[idx, 'battery_soc_percent'] = np.random.uniform(5, 25)
    elif anomaly_type == 'pressure':
        if np.random.rand() > 0.5:
            df.at[idx, 'tank_pressure_bar'] = np.random.uniform(40, 80)
        else:
            df.at[idx, 'tank_pressure_bar'] = np.random.uniform(160, 220)
    elif anomaly_type == 'integrity':
        df.at[idx, 'structural_integrity'] = 0
    elif anomaly_type == 'modules':
        df.at[idx, 'critical_modules_status'] = 0
    elif anomaly_type == 'link':
        df.at[idx, 'telemetry_link_status'] = 0

# Recalcular energy_available_kwh
df['energy_available_kwh'] = (df['battery_voltage_v'] * df['battery_capacity_ah'] * (df['battery_soc_percent'] / 100

# Recalcular launch_decision
def decide_launch(row):
    safe = (
        18 <= row['internal_temp_c'] <= 35 and
        -5 <= row['external_temp_c'] <= 30 and
        46 <= row['battery_voltage_v'] <= 52 and
        20 <= row['battery_current_a'] <= 120 and
        60 <= row['battery_soc_percent'] <= 100 and
```

```
        80 <= row['battery_capacity_ah'] <= 120 and
        5 <= row['power_load_kw'] <= 25 and
        2 <= row['energy_loss_percent'] <= 8 and
        95 <= row['tank_pressure_bar'] <= 145 and
        45 <= row['estimated_autonomy_min'] <= 180 and
        row['structural_integrity'] == 1 and
        row['critical_modules_status'] == 1 and
        row['telemetry_link_status'] == 1
    )
    return 'READY' if safe else 'ABORT'

df['launch_decision'] = df.apply(decide_launch, axis=1)

# Criar a pasta 'telemetria' se não existir
output_dir = './telemetria'
os.makedirs(output_dir, exist_ok=True)

# Salvar CSV na pasta 'telemetria'
output_filepath = os.path.join(output_dir, 'telemetria_aurora_anomalias.csv')
df.to_csv(output_filepath, index=False)
print(f"Dataset gerado com {n_anomalies} anomalias e salvo em {output_filepath}.")
df = pd.read_csv(output_filepath)
display(df.head())
```

```
Dataset gerado com 50 anomalias e salvo em ./telemetria/telemetria_aurora_anomalias.csv.
```

	timestamp	internal_temp_c	external_temp_c	battery_voltage_v	battery_current_a	battery_soc_percent	battery_
0	2026-03-25 02:49:21.488499	30.335580	14.278274	47.131546	24.732245	77.065632	
1	2026-03-25 02:50:21.488499	27.729958	6.708903	48.655491	82.482661	67.851069	
2	2026-03-25 02:51:21.488499	25.152758	18.066490	47.069802	103.057295	76.875116	
3	2026-03-25 02:52:21.488499	33.165332	5.811566	48.427785	34.516320	93.366644	
4	2026-03-25 02:53:21.488499	32.852687	11.313725	51.724188	96.636996	79.684599	

1.2: Algoritmo de Verificação

```
ALGORITMO VerificacaoDecolagemAuroraSiger

ENTRADA: Dataset de Telemetria (CSV)
SAÍDA: Decisão de Lançamento (PRONTO PARA DECOLAR ou DECOLAGEM ABORTADA) para cada linha de telemetria

FUNÇÃO Principal(dataset_telemetria):
  PARA CADA linha NO dataset_telemetria:
    // 1. Interpretar dados de telemetria
    timestamp = linha.timestamp
    internal_temp_c = linha.internal_temp_c
    external_temp_c = linha.external_temp_c
    battery_voltage_v = linha.battery_voltage_v
    battery_current_a = linha.battery_current_a
    battery_soc_percent = linha.battery_soc_percent
    battery_capacity_ah = linha.battery_capacity_ah
    power_load_kw = linha.power_load_kw
    energy_loss_percent = linha.energy_loss_percent
    tank_pressure_bar = linha.tank_pressure_bar
    structural_integrity = linha.structural_integrity
    critical_modules_status = linha.critical_modules_status
    telemetry_link_status = linha.telemetry_link_status
    estimated_autonomy_min = linha.estimated_autonomy_min

    // 2. Calcular energy_available_kwh
    energy_available_kwh = (battery_voltage_v * battery_capacity_ah * battery_soc_percent / 100) / 1000
```

```
    // 3. Construir algoritmo de verificação (Decisão: PRONTO PARA DECOLAR ou DECOLAGEM ABORTADA)
    eh_seguro = VERDADEIRO

    // Verificar faixas de temperatura
    SE NOT (18 <= internal_temp_c <= 35) OU NOT (-5 <= external_temp_c <= 30) ENTÃO
      eh_seguro = FALSO
    FIM SE

    // Verificar faixas da bateria
    SE NOT (46 <= battery_voltage_v <= 52) OU NOT (20 <= battery_current_a <= 120) OU NOT (60 <= battery_soc_percent <
      eh_seguro = FALSO
    FIM SE

    // Verificar faixas de carga e perda de energia
    SE NOT (5 <= power_load_kw <= 25) OU NOT (2 <= energy_loss_percent <= 8) ENTÃO
      eh_seguro = FALSO
    FIM SE

    // Verificar faixa de pressão do tanque
    SE NOT (95 <= tank_pressure_bar <= 145) ENTÃO
      eh_seguro = FALSO
    FIM SE

    // Verificar faixas de autonomia estimada
    SE NOT (45 <= estimated_autonomy_min <= 180) ENTÃO
      eh_seguro = FALSO
    FIM SE

    // Verificar estados de integridade e módulos críticos
    SE structural_integrity == 0 OU critical_modules_status == 0 OU telemetry_link_status == 0 ENTÃO
      eh_seguro = FALSO
    FIM SE

    // Atribuir decisão de lançamento
    SE eh_seguro É VERDADEIRO ENTÃO
      linha.launch_decision = "READY"
    SENÃO
      linha.launch_decision = "ABORT"
    FIM SE

    // Atualizar energy_available_kwh na linha
    linha.energy_available_kwh = energy_available_kwh

  FIM PARA

  RETORNAR dataset_telemetria_atualizado
FIM FUNÇÃO
```

1.3: Script em Python

```
    def verificar_decolagem(df):
        # Recalcular energy_available_kwh
        df["energy_available_kwh"] = (df["battery_voltage_v"] * df["battery_capacity_ah"] * (df["battery_soc_percent"] /

        def decide_launch(row):
            safe = (
                (18 <= row["internal_temp_c"] <= 35) and
                (-5 <= row["external_temp_c"] <= 30) and
                (46 <= row["battery_voltage_v"] <= 52) and
                (20 <= row["battery_current_a"] <= 120) and
                (60 <= row["battery_soc_percent"] <= 100) and
                (80 <= row["battery_capacity_ah"] <= 120) and
                (5 <= row["power_load_kw"] <= 25) and
                (2 <= row["energy_loss_percent"] <= 8) and
```

```
            (95 <= row["tank_pressure_bar"] <= 145) and
            (45 <= row["estimated_autonomy_min"] <= 180) and
            (row["structural_integrity"] == 1) and
            (row["critical_modules_status"] == 1) and
            (row["telemetry_link_status"] == 1)
        )
        return "PRONTO PARA DECOLAR" if safe else "ABORTAR MISSÃO"

    df["launch_decision"] = df.apply(decide_launch, axis=1)
    return df

if __name__ == "__main__":
    # Carregar o dataset com anomalias
    df_telemetria = pd.read_csv("./telemetria/telemetria_aurora_anomalias.csv")

    # Executar a verificação
    df_resultado = verificar_decolagem(df_telemetria)

    # Imprimir apenas o resultado final (contagem de READY/ABORT)
    print("Resultado Final:")
    counts = df_resultado["launch_decision"].value_counts()
    for label, count in counts.items():
        print(f"{label}    {count}")

    # Opcional: Salvar o dataset com as decisões atualizadas
    df_resultado.to_csv("./telemetria/telemetria_aurora_decisoes.csv", index=False)
    print("Resultados salvos no arquivo 'telemetria_aurora_decisoes.csv'")
```

```
Resultado Final:
PRONTO PARA DECOLAR    450
ABORTAR MISSÃO    50
Resultados salvos no arquivo 'telemetria_aurora_decisoes.csv'
```

1.4: Análise energética

```
# Configurações
n_rows = 500
start_time = datetime.now()

# Gerar timestamps sequenciais (intervalo de 1 minuto)
timestamps = [start_time + timedelta(minutes=i) for i in range(n_rows)]

# Gerar dados aleatórios dentro das faixas especificadas
data = {
    'timestamp': timestamps,
    'internal_temp_c': np.random.uniform(18, 35, n_rows),
    'external_temp_c': np.random.uniform(-5, 30, n_rows),
    'battery_voltage_v': np.random.uniform(46, 52, n_rows),
    'battery_current_a': np.random.uniform(20, 120, n_rows),
    'battery_soc_percent': np.random.uniform(60, 100, n_rows),
    'battery_capacity_ah': np.random.uniform(80, 120, n_rows),
    'power_load_kw': np.random.uniform(5, 25, n_rows),
    'energy_loss_percent': np.random.uniform(2, 8, n_rows),
    'tank_pressure_bar': np.random.uniform(95, 145, n_rows),
    'estimated_autonomy_min': np.random.uniform(45, 180, n_rows),
    'structural_integrity': [1] * n_rows,
    'critical_modules_status': [1] * n_rows,
    'telemetry_link_status': [1] * n_rows
}

df = pd.DataFrame(data)

# Cálculo: energy_available_kwh = (battery_voltage_v * battery_capacity_ah * battery_soc_percent/100) / 1000
df['energy_available_kwh'] = (df['battery_voltage_v'] * df['battery_capacity_ah'] * (df['battery_soc_percent'] / 100

# Decisão: READY se todas condições seguras forem atendidas, senão ABORT
# Como os valores gerados estão dentro das faixas seguras e os estados são fixos em 1, a decisão será READY.
# No entanto, para garantir a lógica, vamos aplicar a regra.
def decide_launch(row):
    # Critérios de segurança baseados nas faixas fornecidas
    safe = (
        18 <= row['internal_temp_c'] <= 35 and
        -5 <= row['external_temp_c'] <= 30 and
        46 <= row['battery_voltage_v'] <= 52 and
```

```
        20 <= row['battery_current_a'] <= 120 and
        60 <= row['battery_soc_percent'] <= 100 and
        80 <= row['battery_capacity_ah'] <= 120 and
        5 <= row['power_load_kw'] <= 25 and
        2 <= row['energy_loss_percent'] <= 8 and
        95 <= row['tank_pressure_bar'] <= 145 and
        45 <= row['estimated_autonomy_min'] <= 180 and
        row['structural_integrity'] == 1 and
        row['critical_modules_status'] == 1 and
        row['telemetry_link_status'] == 1
    )
    return 'READY' if safe else 'ABORT'

df['launch_decision'] = df.apply(decide_launch, axis=1)

# Reordenar colunas conforme solicitado
cols = [
    'timestamp', 'internal_temp_c', 'external_temp_c', 'battery_voltage_v',
    'battery_current_a', 'battery_soc_percent', 'battery_capacity_ah',
    'energy_available_kwh', 'power_load_kw', 'energy_loss_percent',
    'tank_pressure_bar', 'structural_integrity', 'critical_modules_status',
    'telemetry_link_status', 'estimated_autonomy_min', 'launch_decision'
]
df = df[cols]

# Salvar para CSV
df.to_csv('./telemetria/telemetria_sintetica.csv', index=False)
print("Dataset gerado com sucesso: telemetria_sintetica.csv")
```

```
Dataset gerado com sucesso: telemetria_sintetica.csv
```

```
# Carregar o dataset gerado na seção 1.4
df_energia = pd.read_csv('./telemetria/telemetria_sintetica.csv')

print("Primeiras 5 linhas do dataset de análise energética:")
display(df_energia.head())
```

```
Primeiras 5 linhas do dataset de análise energética:
```

	timestamp	internal_temp_c	external_temp_c	battery_voltage_v	battery_current_a	battery_soc_percent	battery_
0	2026-03-25 02:49:59.865246	19.287346	3.539443	51.728678	44.410377	60.317703	
1	2026-03-25 02:50:59.865246	18.047543	-0.671049	48.804799	106.087956	96.872499	
2	2026-03-25 02:51:59.865246	28.553214	23.972808	48.370257	94.151140	80.526049	
3	2026-03-25 02:52:59.865246	23.544284	5.824117	49.033525	113.841078	99.444213	
4	2026-03-25 02:53:59.865246	25.055983	22.093357	48.845152	52.299194	89.337979	

```
print("Estatísticas descritivas para as variáveis de energia:")
energy_cols = ['battery_voltage_v', 'battery_current_a', 'battery_soc_percent', 'battery_capacity_ah', 'energy_avail
display(df_energia[energy_cols].describe())
```

```
Estatísticas descritivas para as variáveis de energia:
```

	battery_voltage_v	battery_current_a	battery_soc_percent	battery_capacity_ah	energy_available_kwh	power_loa
count	500.000000	500.000000	500.000000	500.000000	500.000000	500.00
mean	48.974999	71.761490	80.279604	100.169417	3.941327	15.04
std	1.716339	29.489596	11.881449	11.143159	0.759566	5.70
min	46.008022	20.043194	60.010144	80.017657	2.422442	5.02
25%	47.549866	46.384597	69.848062	90.730126	3.318176	10.16
50%	48.796519	71.754366	80.943001	99.097979	3.913118	14.86
75%	50.509085	100.022050	90.637748	109.590586	4.505422	19.95
max	51.992979	119.979031	99.919530	119.908042	5.772316	24.98

Análise da Energia Disponível e Carga de Potência

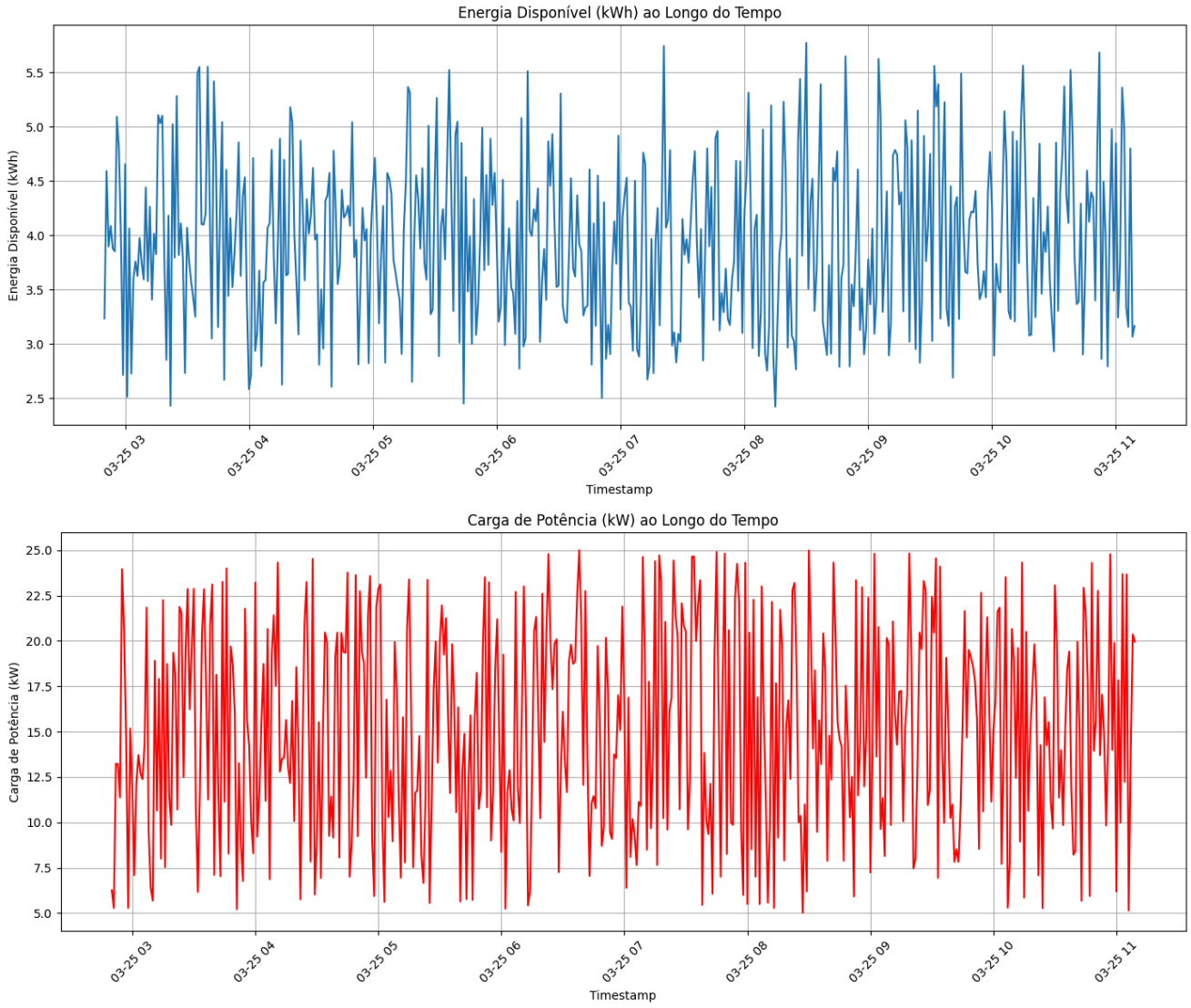
Vamos visualizar a distribuição da energia disponível e da carga de potência ao longo do tempo.

```
import matplotlib.pyplot as plt
import seaborn as sns

df_energia['timestamp'] = pd.to_datetime(df_energia['timestamp'])

plt.figure(figsize=(14, 6))
sns.lineplot(x='timestamp', y='energy_available_kwh', data=df_energia)
plt.title('Energia Disponível (kWh) ao Longo do Tempo')
plt.xlabel('Timestamp')
plt.ylabel('Energia Disponível (kWh)')
plt.grid(True)
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()

plt.figure(figsize=(14, 6))
sns.lineplot(x='timestamp', y='power_load_kw', data=df_energia, color='red')
plt.title('Carga de Potência (kW) ao Longo do Tempo')
plt.xlabel('Timestamp')
plt.ylabel('Carga de Potência (kW)')
plt.grid(True)
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```



Double-click (or enter) to edit

1.5: Análise Assistida por IA

```
import google.generativeai as genai
import os

# 🔑 Chave de API configurada
GEMINI_API_KEY = "AIzaSyD4lwKkhWFEmNqcZHW-ZWapnqLYYJWQ2ds"

try:
    genai.configure(api_key=GEMINI_API_KEY)

    # 🔍 Busca dinâmica pelo modelo disponível
    print("Buscando modelos compatíveis...")
```

```
    available_models = [m.name for m in genai.list_models() if 'generateContent' in m.supported_generation_methods]

    if not available_models:
        print("Erro: Nenhum modelo de geração de conteúdo disponível para esta chave.")
    else:
        # Tenta priorizar modelos flash ou pro, senão pega o primeiro da lista
        model_name = next((m for m in available_models if "flash" in m), available_models[0])
        print(f"Modelo selecionado: {model_name}")

        model = genai.GenerativeModel(model_name)

        path_df = "/content/telemetria/telemetria_aurora_anomalias.csv"
        if os.path.exists(path_df):
            df = pd.read_csv(path_df)

            resumo = {
                "colunas": list(df.columns),
                "estatisticas": df.describe(include='all').fillna("").to_dict(),
                "amostra": df.head(10).to_dict()
            }

            prompt = f"""
            Você é um analista de dados especialista em telemetria espacial.
            Analise os dados da nave Aurora Siger:
            1. Classificação técnica dos parâmetros.
            2. Identificação de anomalias críticas.
            3. Sugestões de mitigação de risco.

            DADOS:
            {resumo}
            """

            print(f"Solicitando análise ao {model_name}...\n")
            response = model.generate_content(prompt)

            print("--- RESULTADO DA ANÁLISE (GEMINI) ---")
            print(response.text)
        else:
            print(f"Erro: Arquivo {path_df} não encontrado.")

except Exception as e:
    print(f"Ocorreu um erro: {e}")
```